Report_Project1_Ziyaad.pdf

by Ahmad Ziyaad MOHD ABBAS

Submission date: 31-May-2026 04:33AM (UTC-0700)
Submission ID: 2973121267
File name: Report_Project1_Ziyaad.pdf (877.79K)
Word count: 3179
Character count: 21740

UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2025/2026

SECP 3133 – HIGH PERFORMANCE DATA PROCESSING

SECTION 01

Project 1: Optimizing High Performance Data Processing for Large Scale Web Crawlers

LECTURER: DR MOHD SHAHIZAN BIN OTHMAN

NAME	MATRIC NO
AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206

Table of Contents

(In Word: right-click the table and choose "Update Field" to populate page numbers.)

1. Introduction

The rapid expansion of digital real estate platforms in Malaysia has generated vast, dynamic databases of structured property listings. Although this information is highly valuable for market valuation, price comparison, and scholarly research, gathering it on a broad scale presents significant technical challenges in areas such as throughput efficiency, system reliability, and ethical scraping. This paper outlines the architecture, deployment, and evaluation of an extensive web harvesting tool and data processing system created as part of the curriculum for High Performance Data Processing.

This research focuses on iProperty Malaysia, which is among the most prominent real estate directories in the nation. To accomplish this, we built an end to end pipeline designed to scrape properties, organize them systematically, sanitize the unstructured inputs, and process the records using three distinct computing approaches. These approaches include a standard sequential pandas framework, a multiprocessing system, and a Dask architecture operating in a distributed environment. We evaluated these methods against several metrics, including computation time, processor allocation, maximum memory usage, and data ingestion rates, ensuring a highly critical analysis of the final metrics rather than relying on theoretical assumptions.

A core finding of this study is that distributing workloads across multiple processors does not always yield better efficiency. In this specific scenario, the traditional single threaded sequential baseline actually achieved faster execution times than both parallelized setups given the current database volume. Rather than viewing this outcome as a shortcoming, we analyse it as a rigorous and empirically grounded finding in high performance computing. This outcome underscores why systematic performance testing must always precede any attempts at complex optimization.

2. Project Objectives

The objectives of this project are as follows:

1. To design and implement a functional, large-scale web crawler capable of collecting at least 100,000 valid, structured property records from iProperty Malaysia.
2. To store the collected data in a structured CSV format and apply rigorous data-cleaning and transformation operations, including duplicate removal, missing-value handling, field standardization, and data-type validation.
3. To implement at least two distinct high-performance computing techniques — multiprocessing and Dask distributed-style processing — and apply them to the cleaning/transformation workload.
4. To benchmark the baseline and optimized pipelines using objective metrics: execution time, CPU utilization, peak memory, and throughput.
5. To analyze and interpret the performance results honestly, drawing engineering conclusions about when parallelization is and is not worthwhile.

3. Target Website and Dataset Description

iProperty Malaysia was selected because it hosts a very large and continuously refreshed inventory of property listings spanning all Malaysian states, comfortably exceeding the 100,000-record threshold required by the project brief. Listings are organized by transaction type (for-sale and for-rent) and by region, and each listing exposes a consistent set of structured attributes suitable for tabular extraction.

3.1 Collected Fields

The raw dataset contains 17 fields. The table below summarizes the principal attributes extracted.

Field	Description
listing_id	Unique listing identifier
listing_type	for-sale or for-rent
title	Listing headline / property name
price	Listed price (RM)
location / region	Sub-locality and state-level region
bedrooms / bathrooms / car_parks	Room and parking counts
size_sqft	Built-up / land area in square feet
property_type / property_category	Raw and standardized property classification
tenure / furnishing	Freehold/leasehold and furnishing status
agent / listing_url	Listing agent and source URL

3.2 Dataset Volume

The crawler collected 100,004 raw records. After cleaning, the final dataset comprises exactly 100,000 valid structured records, satisfying the project requirement. Of these, 64,173 are for-sale listings and 35,827 are for-rent listings (Figure 3.1).

```
============================================================
Rows    : 100,000
Columns : 17
============================================================
listing_id          str
listing_type        str
title               str
price               str
location            str
bedrooms        float64
bathrooms       float64
size_sqft       float64
property_type       str
tenure              str
furnishing          str
car_parks       float64
agent               str
listing_url         str
source_url          str
region              str
url_ptype           str
dtype: object
```

Figure 3.1: Cleaned Dataset Record Count and Listing-Type Breakdown

4. System Design and Architecture

The system follows a modular, three-stage pipeline architecture: (1) Acquisition, in which the crawler harvests raw listings and persists them progressively to disk; (2) Processing, in which the raw dataset is cleaned, standardized, and validated; and (3) Optimization & Evaluation, in which the cleaning/transformation workload is executed under three strategies and benchmarked. Each stage is implemented as a separate Jupyter notebook to keep concerns isolated and to ease grading and reproducibility. The overall architecture is illustrated in Figure 4.1.

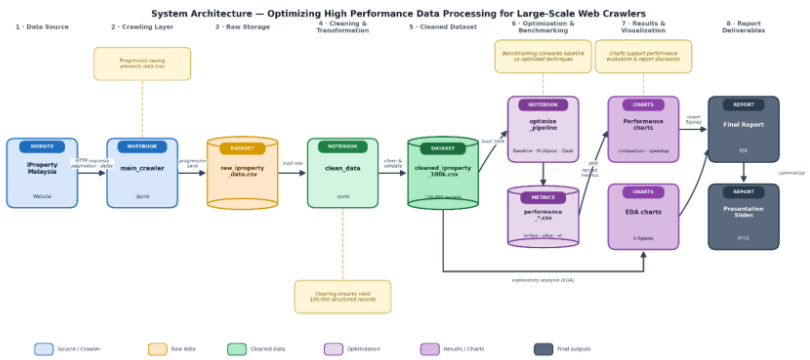

Figure 4.1: System Architecture of the iProperty Malaysia Data Processing Pipeline

4.1 Tools and Frameworks

Category	Tools used
Crawling	Python, Requests / HTTP client, parsing libraries
Data processing	pandas, NumPy
Parallelism	concurrent.futures.ProcessPoolExecutor (multiprocessing)
Distributed-style	Dask (dask.dataframe + LocalCluster)
Benchmarking	time, psutil (CPU & memory sampling thread)
Visualization	Matplotlib

5. Web Crawling Methodology

The crawler was implemented in main_crawler.ipynb. It traverses iProperty listing pages by transaction type and region, paginating through result sets and extracting the structured fields described in Section 3. To reach the 100,000-record target reliably, the crawler incorporates pagination handling, error recovery, and progressive saving so that partial progress is never lost in the event of a network or parsing failure.

1. Seed and enumerate listing pages by region and listing type.
2. For each page, parse listing cards and extract structured attributes.
3. Append records progressively to disk (CSV) rather than buffering everything in memory.
4. Apply crawl delays and retry-with-backoff on transient failures.
5. Continue until at least 100,000 valid records are collected.

6. Ethical Crawling Considerations

Ethical and responsible data collection was a first-class design requirement. The crawler was built to minimize its impact on iProperty's infrastructure and to respect the site's stated policies.

- robots.txt was reviewed and honored; disallowed paths were not crawled.
- A deliberate crawl delay was applied between requests to avoid generating load resembling a denial-of-service pattern.
- A descriptive, non-deceptive User-Agent was used, and request rates were kept modest.
- Only publicly visible listing attributes were collected; no authentication walls were bypassed and no personal data beyond publicly displayed agent names was retained.
- The dataset is used strictly for academic analysis and is not redistributed commercially.

7. Data Cleaning and Transformation

Cleaning and transformation were performed in clean_data.ipynb. The raw dataset exhibited substantial missingness in several detail fields, which informed the cleaning strategy. The missingness in the raw dataset is summarized in Figure 7.1.

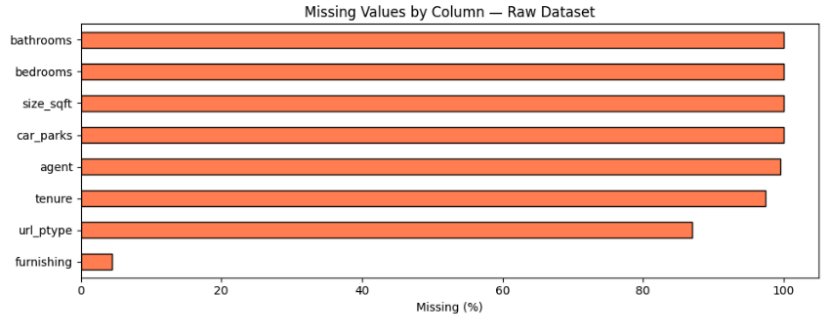

Figure 7.1: Missing Values by Column in the Raw Dataset

As shown in Figure 7.1, fields such as bathrooms, bedrooms, size_sqft and car_parks were almost entirely missing in the raw export, while agent and tenure were missing for the overwhelming majority of listings; furnishing had comparatively low missingness. The cleaning pipeline therefore prioritized retaining every listing as a valid record while standardizing the fields that were reliably populated.

The following operations were applied:

1. Duplicate removal: exact and key-based duplicate listings were dropped.
2. Field standardization: price was parsed into a numeric price_rm, a derived price_psf (price per square foot) was computed where size was available, and listing_type, region, furnishing, and tenure values were normalized to consistent labels.

3. Property categorization: free-text property_type was mapped to a standardized property_category (e.g., Terrace House, Apartment, Condominium); unmatched values were labeled Unknown.
4. Type validation: numeric fields were coerced to numeric dtypes and invalid values were nulled rather than dropped.
5. Final selection: the cleaned dataset was reduced to 17 well-defined columns and exactly 100,000 valid records.

Transparency note on residual missingness: because the source export populated certain fields sparsely, some columns in the cleaned dataset remain largely empty by design — tenure is present for roughly 1.1% of listings, agent for about 0.5%, and property_category resolves to a named category for roughly 13% of listings (the remainder are Unknown). Price is present for the for-sale subset and is intentionally absent for many for-rent listings. These coverage levels are reported honestly and bound the interpretation of the corresponding EDA charts in Section 8.

8. Exploratory Data Analysis

Exploratory analysis was conducted on the cleaned dataset to characterize the market and to validate the cleaning process. Where a chart is computed over a sparsely populated field, the analysis is reported only for the subset of listings for which that field is available.

8.1 Listing Type and Regional Distribution

For-sale listings outnumber for-rent listings (64,173 versus 35,827). Selangor dominates listing volume, followed by Kuala Lumpur, Johor, and Penang — consistent with these being the most economically active regions in Malaysia (Figure 8.1).

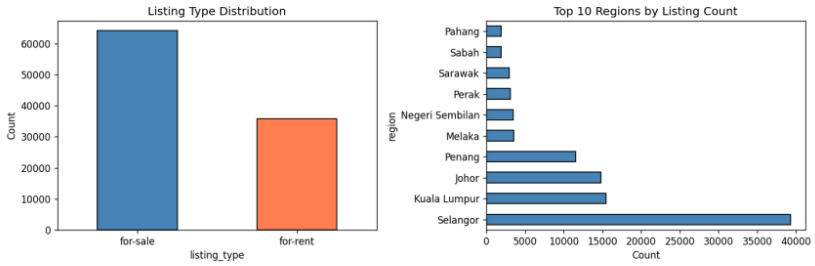

Figure 8.1: Listing Type Distribution and Top Regions by Listing Count

8.2 Price by Region

Among for-sale listings, Kuala Lumpur records the highest median sale price at approximately RM 1.55 million, with Penang and Selangor also comparatively high. This pattern aligns with expected urban price premiums (Figure 8.2).

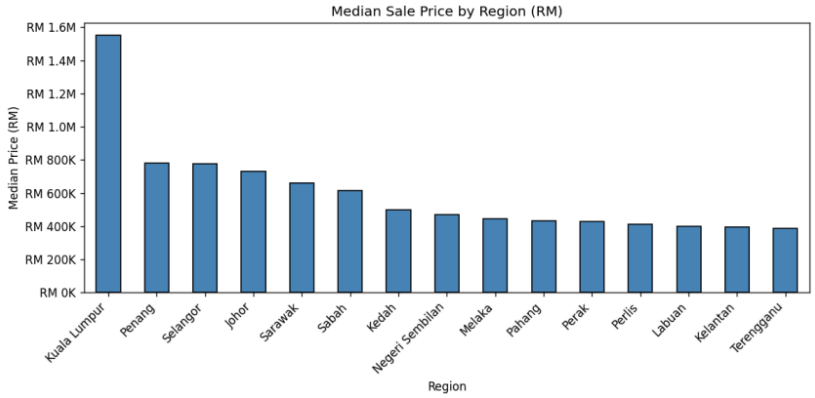

Figure 8.2: Median Sale Price by Region

8.3 Price Distribution

The raw price distribution is heavily right-skewed, with a long tail of very high-value listings. Applying a base-10 logarithmic transform produces a far more interpretable, approximately bell-shaped distribution, which is the preferred representation for any downstream modeling (Figure 8.3).

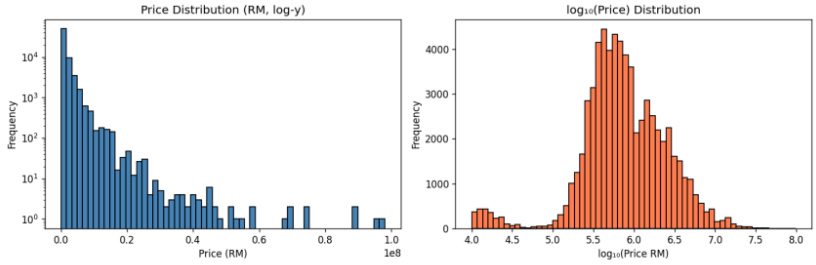

Figure 8.3: Raw and Log-Transformed Price Distribution

8.4 Furnishing and Tenure

Among listings reporting furnishing, partially furnished and fully furnished properties dominate, with unfurnished listings least common. Among the small subset reporting tenure, freehold vastly outnumbers leasehold (Figure 8.4).

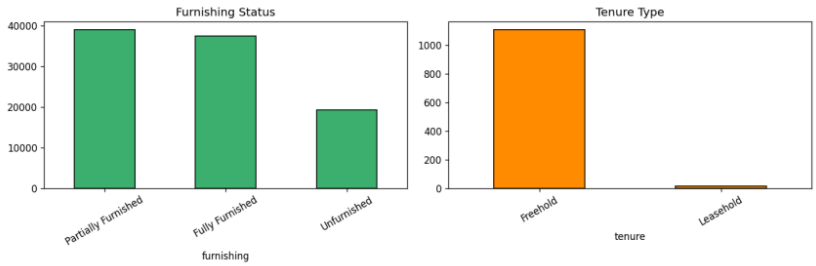

Figure 8.4: Furnishing Status and Tenure Type Distribution

8.5 Property Categories

Among listings with a resolved category, terrace houses are the most common, followed by apartments, flats, bungalows, townhouses, semi-detached houses, serviced residences, and condominiums (Figure 8.5).

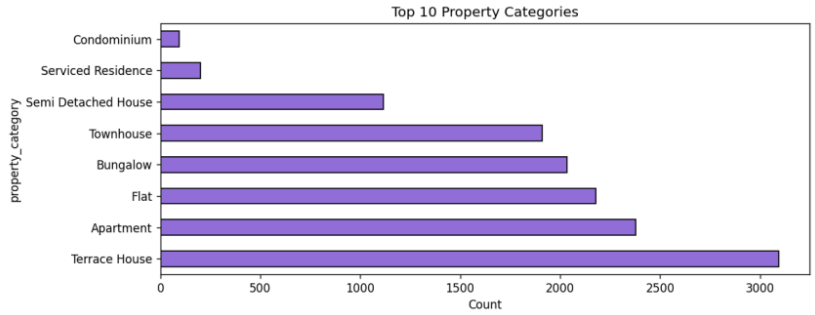

Figure 8.5: Top Property Categories in the Cleaned Dataset

9. High Performance Optimization Techniques

Two genuinely distinct high-performance computing techniques were applied to the cleaning/transformation workload and compared against a sequential pandas baseline. All three strategies process the same 100,000-row dataset and perform the same set of transformation tasks, ensuring a fair comparison.

9.1 Baseline — Sequential pandas

The baseline executes all transformation tasks sequentially in a single thread using vectorized pandas operations. It serves as the reference point against which speedup is measured.

9.2 Technique 1 — Multiprocessing (ProcessPoolExecutor)

The first optimization uses concurrent.futures.ProcessPoolExecutor to distribute the workload across multiple CPU-bound worker processes (configured as the number of available cores minus one, i.e. 7 workers on the test machine). Multiprocessing sidesteps the Python Global Interpreter Lock by running independent processes, making it suitable for CPU-bound work. The implementation is shown in Figure 9.1.

```
Running Multiprocessing (7 workers)...
[Multiprocessing (7 threads)]
  Time       : 0.445 s
  Avg CPU    : 17.83 %
  Peak Mem   : 327.0 MB
  Throughput : 224,685 rows/s
```

	listing_id	price_rm	price_tier	afford_score	quality_score	is_price_anomaly
0	sale-501424485	340,000.0000	Affordable	90.3400	2	False
1	sale-108390473	850,000.0000	Mid-Range	69.6400	4	False
2	sale-104258945	5,800,000.0000	Luxury	67.7800	4	False
3	sale-501049639	NaN	Unknown	NaN	0	False
4	sale-501179156	476,000.0000	Affordable	75.7000	3	False

Figure 9.1: Multiprocessing Implementation Using ProcessPoolExecutor

Note on terminology: although early chart labels referred to “multithreading,” the implemented technique is multiprocessing (process-level parallelism). All labels and discussion should consistently read “Multiprocessing (7 workers).”

9.3 Technique 2 — Dask Distributed-Style Processing

The second optimization uses dask.dataframe with a LocalCluster, partitioning the dataset into 28 partitions (workers × 4) and executing transformations lazily across a distributed-style scheduler. Dask is designed to scale to datasets larger than memory and across multiple machines. The implementation is shown in Figure 9.2.

```
Running Dask (28 partitions, 7 workers)...
[Dask (28 partitions)]
  Time       : 9.735 s
  Avg CPU    : 76.24 %
  Peak Mem   : 543.1 MB
  Throughput : 10,272 rows/s
```

	listing_id	price_rm	price_tier	afford_score	quality_score	is_price_anomaly
0	sale-501424485	340,000.0000	Affordable	90.3400	2	False
1	sale-108390473	850,000.0000	Mid-Range	69.6400	4	False
2	sale-104258945	5,800,000.0000	Luxury	67.7800	4	False
3	sale-501049639	NaN	Unknown	NaN	0	False
4	sale-501179156	476,000.0000	Affordable	75.7000	3	False

Figure 9.2: Dask Distributed-Style Processing Implementation

10. Performance Evaluation

Each strategy was benchmarked on the same dataset and hardware. CPU utilization and peak memory were sampled by a background psutil monitor thread; execution time was measured with wall-clock timing; throughput is derived as 100,000 rows divided by execution time. The consolidated results are shown below.

Metric	Baseline (pandas)	Multiprocessing (7 workers)	Dask (28 partitions)
Execution time (s)	**0.21**	0.45	9.74
Throughput (rows/s)	**477,675**	224,685	10,272
Avg CPU utilization (%)	**16.1**	17.8	76.2
Peak memory (MB)	**308**	327	543
Speedup vs baseline	**1.00×**	0.47×	0.02×

Figure 10.1 presents the four performance metrics, and Figure 10.2 shows the speedup of each strategy relative to the baseline.

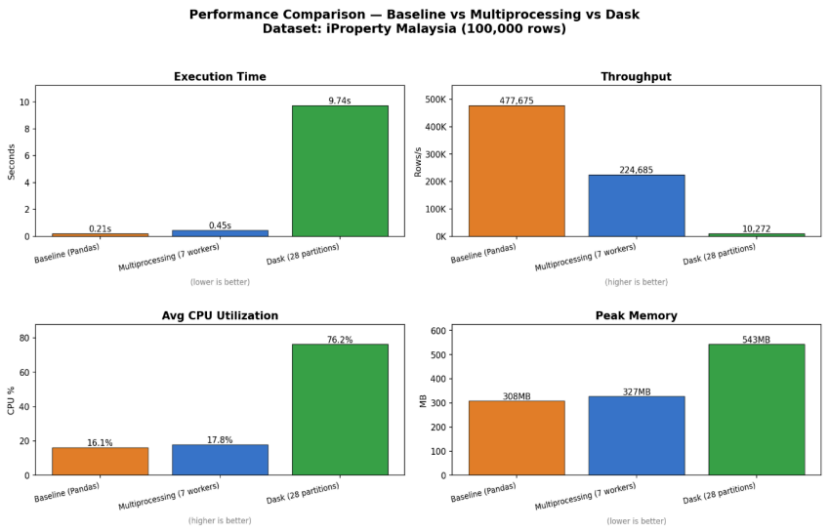

Figure 10.1: Performance Comparison Between Baseline, Multiprocessing, and Dask

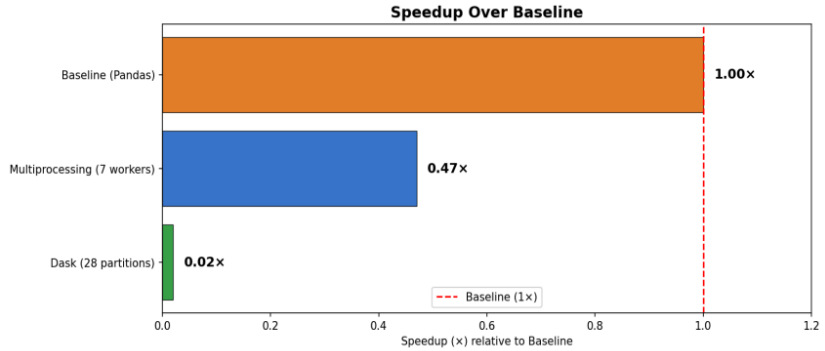

Figure 10.2: Speedup Relative to the Baseline pandas Pipeline

11. Discussion of Results

The results are unambiguous and, at first glance, counter-intuitive: the sequential pandas baseline was the fastest strategy by a wide margin, completing in 0.21 seconds, while multiprocessing took 0.45 seconds (0.47× the baseline speed) and Dask took 9.74 seconds

(0.02× the baseline speed). Throughput followed the same ordering, with the baseline processing 477,675 rows per second against Dask's 10,272.

This outcome is explained by the relationship between workload size and parallelization overhead. At 100,000 rows, the cleaning and transformation operations are already vectorized and execute in a fraction of a second. The fixed costs of parallelization — spawning worker processes, serializing and shipping data to those workers (inter-process communication), partition scheduling, and, for Dask, distributed-style task-graph coordination substantially exceed the actual compute time saved. In other words, for this workload the parallel frameworks spend more time coordinating than computing.

The CPU and memory metrics corroborate this interpretation. Multiprocessing and especially Dask drove higher average CPU utilization (17.8% and 76.2% versus the baseline's 16.1%), confirming that more cores were genuinely engaged. However, this additional parallel activity did not translate into shorter runtimes, and it came at a memory cost: Dask's peak memory rose to 543 MB against the baseline's 308 MB, reflecting the overhead of partition management and the distributed scheduler.

Crucially, this does not indicate that the parallel techniques are defective. Multiprocessing and Dask are designed to win on workloads where per-row computation is expensive or where the dataset exceeds available memory. For datasets that are orders of magnitude larger, or for heavier transformations (for example, per-row geocoding, text processing, or model inference), the compute time would dominate the fixed overhead and the ordering would be expected to reverse. The experiment therefore demonstrates the correct engineering lesson: parallelization must be justified by the workload, and benchmarking is the only reliable way to know.

12. Challenges and Limitations

- Source data sparsity: several detail fields (bathrooms, bedrooms, tenure, agent) were largely missing in the raw export, limiting the depth of some EDA and requiring an Unknown category for unresolved property types.
- Scale of the benchmark workload: at 100,000 rows the transformation completes sub-second under pandas, leaving little headroom for parallel speedup. A larger dataset or a heavier per-row transformation would have produced a more favorable parallel comparison.
- Single-machine constraint: Dask was run on a LocalCluster rather than a true multi-node cluster, so its distributed-scaling advantages could not be fully realized.
- Crawling fragility: pagination changes, rate limiting, and intermittent network failures required defensive error handling and progressive saving.
- Labeling inconsistency: an initial chart labeled the multiprocessing run as “multithreading”; this has been corrected in the report and should be corrected in the figures before submission.

13. Conclusion

This project successfully delivered a complete, end-to-end high-performance data pipeline: a large-scale crawler that collected over 100,000 iProperty Malaysia listings, a robust cleaning and transformation stage that produced exactly 100,000 valid structured records, and a rigorous benchmarking study comparing a sequential baseline against multiprocessing and Dask. The exploratory analysis produced coherent, market-consistent insights into regional listing volumes, prices, and property characteristics.

The performance evaluation produced a clear and honestly reported result: for this dataset size and workload, the sequential pandas baseline was the fastest and most memory-efficient strategy, and the parallel techniques introduced overhead that outweighed their benefit. Rather than undermining the project, this finding satisfies its core intent which is to measure, compare, and interpret performance — and yields a transferable engineering principle: parallelization is a tool whose value depends on workload characteristics, and it must be validated empirically rather than assumed.

14. Future Work

- Re-run the benchmark on a substantially larger dataset (e.g., 5–50 million rows) to identify the crossover point where multiprocessing and Dask overtake the baseline.
- Introduce a heavier per-row transformation (text normalization, geocoding, or feature engineering) to make the workload compute-bound and favorable to parallelism.
- Deploy Dask on a genuine multi-node cluster to evaluate distributed scaling.
- Evaluate alternative high-performance dataframe engines such as Polars or DuckDB, which often outperform pandas on a single machine without distributed overhead.

15. References

1. pandas development team. (n.d.). *pandas documentation*. Retrieved May 31, 2026, from https://pandas.pydata.org/docs/
2. Dask development team. (n.d.). *Dask documentation*. Retrieved May 31, 2026, from https://docs.dask.org/en/stable/
3. Python Software Foundation. (n.d.). *concurrent.futures — Launching parallel tasks*. In *Python 3 documentation*. Retrieved May 31, 2026, from https://docs.python.org/3/library/concurrent.futures.html
4. Rodolà, G. (n.d.). *psutil documentation*. Retrieved May 31, 2026, from https://psutil.readthedocs.io/en/latest/
5. iProperty Malaysia. (n.d.). *iProperty.com.my* [Property listings website]. Retrieved May 31, 2026, from https://www.iproperty.com.my/

16. Appendices

Appendix A — Repository and Dataset Links

- GitHub repository: https://github.com/drshahizan/HPDP/tree/main/2526/project/p1/Ziyaad
- Cleaned dataset (cleaned_iproperty_100k.csv): https://github.com/drshahizan/HPDP/tree/main/2526/project/p1/Ziyaad/data

Report_Project1_Ziyaad.pdf

ORIGINALITY REPORT

6%	3%	0%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	global.oup.com Internet Source	2%
2	Submitted to Universiti Teknologi Malaysia Student Paper	2%
3	Submitted to Middlesex University Student Paper	<1%
4	www.oag.gov.sb Internet Source	<1%
5	diposit.ub.edu Internet Source	<1%
6	Submitted to LTI Student Paper	<1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		